

UTM

UNIVERSITI TEKNOLOGI MALAYSIA

SCSE2233-03: Software Design and Architecture • SEM 2025/2026-2

Online Food Delivery System

Submission Date: May 2026

Group Name	Hub
Course	SCSE2233: Software Design and Architecture
Semester	Semester 2, 2025/2026
Section	Section 3
Lecturer	MUHAMMAD KHATIBSYARBINI

Group : 11

Group Member	Matric Number
Mohammed Mohsen Mohammed Alsakkaf	**A23CS4026**
Imtiaj Ahmed	A24CS4017
Anas Ahmed Hisham	**A24CS4006**

Table of Content

1. Executive Summary 3
2. Case Study Selection [REQ-01] 3
2.1. Problem Statement 3
2.2. System Scope & Identified Use Cases 4
2.3. Key Non-Functional Requirements 4
3. Use Case Diagram [REQ-02] 5
3.1. System Actors 5
3.2. Use Case Diagram 5
3.3. Actor-Use Case Associations Summary 6
4. Use Case Specifications [REQ-03] 7
4.1. Use Case Specification — User Login / Registration | Module 1 | Mohammed Alsakkaf 7
4.2. Use Case Specification — Browse Restaurants & Menu | Module 1 | Mohammed Alsakkaf 9
4.3. Use Case Specification — Add Items to Cart | Module 2 | Anas Ahmed Hisham 11
4.4. Use Case Specification — Place Order & Confirm | Module 2 | Anas Ahmed Hisham 13
4.5. Use Case Specification — Process Payment | Module 3 | Imtiaj 15
4.6. Use Case Specification — Track Delivery | Module 3 | Imtiaj 17
5. Sequence Diagrams [REQ-04] 19
5.1. Sequence Diagram — User Login & Browse Restaurants | Module 1 | Mohammed Alsakkaf 19
5.2. Sequence Diagram — Add to Cart & Place Order | Module 2 | Anas Ahmed Hisham 20
5.3. Sequence Diagram — Process Payment & Track Delivery | Module 3 | Imtiaj 21
6. Architectural Style [REQ-05] 22
6.1. Selected Architectural Style: Microservices Architecture 22
6.2. Rationale — Why Microservices? 22
6.3. Mapping to Functional Requirements 23
6.4. Trade-offs and Mitigations 23
7. UML Structural Diagrams [REQ-06] 24
7.1. Class Diagram — User Auth & Restaurant Browsing Module | Module 1 | Mohammed Alsakkaf 24
7.1.1. Coherence Validation — Module 1 25
7.2. Class Diagram — Shopping Cart & Order Placement Module | Module 2 | Anas Ahmed Hisham 26
7.2.1. Coherence Validation — Module 2 27
7.3. Class Diagram — Payment Processing & Delivery Tracking Module | Module 3 | Imtiaj 28
7.3.1. Coherence Validation — Module 3 29
8. Conclusion & Reflection 30
8.1. Summary 30
8.2. Lessons Learned 30
9. Appendix 31
9.1. Module Assignment Matrix 31
9.2. Group Work Contributions 31

1. Executive Summary

This report presents a complete Software Architecture Design for an Online Food Delivery System — a real-world platform enabling customers to discover restaurants, place food orders, process payments, and track deliveries in real time. The system is comparable in scope and complexity to platforms such as Grab Food, Foodpanda, and Uber Eats

The team selected a Microservices Architectural Style to address the system's critical non-functional requirements: independent scalability of high-traffic modules (e.g., search and payment), fault isolation between services, and the ability for separate development teams to own and deploy individual services. The architecture decomposes the system into three primary modules owned by individual team members, each fully specified through use case documentation, interaction sequence diagrams, and UML structural models.

Module	Scope	Owner
Module 1	User Authentication & Restaurant Browsing	Mohammed Alsakkaf
Module 2	Shopping Cart & Order Placement	Anas Ahmed Hisham
Module 3	Payment Processing & Delivery Tracking	Imtiaj

2. Case Study Selection [REQ-01]

2.1. Problem Statement

The **Online Food Delivery System** is a multi-actor platform meeting urban demand for reliable food services by enabling real-time ordering, secure payments, and live tracking. This domain was selected for its complex architectural trade-offs, requiring **high read throughput** for searches, **strict consistency** for financial transactions, and **millisecond-level performance** across a distributed user base, making it an ideal study for modern software design.

2.2. System Scope & Identified Use Cases

The system encompasses the following primary use cases, sufficient in number and interdependency to justify deep architectural analysis:

Use Case	Module	Owner	Description
User Registration & Login	1	Mohammed Alsakkaf	Handles account creation, credential validation, and secure session management.
Browse Restaurants & Search Menu Items			Allows users to filter restaurants by location/cuisine and view digital menus.
Add Items to Cart & Manage Cart	2	Anas Ahmed Hisham	Manages temporary selection of food items, quantities, and price calculations.
Place Order & Order Confirmation			Finalizes the selection, validates inventory, and converts the cart into an active order.
Process Payment via Payment Gateway	3	Imtiaj	Securely handles financial transactions and interfacing with external payment providers.
Track Delivery in Real Time			Provides live GPS updates and status transitions from the kitchen to the doorstep.
Manage Restaurant Listings	Admin	All	Administrative tool for onboarding new partners and updating restaurant data.
View Sales Analytics Dashboard			Provides high-level business intelligence on revenue, order volume, and trends.
Manage Delivery Fleet Assignment			Oversees driver dispatching and logistics coordination.

2.3. Key Non-Functional Requirements

NFR	Requirement	Target
Scalability	Handle 50,000 concurrent users during peak hours (e.g., lunch/dinner rush)	Auto-scale search service 5x in <2 min
Performance	Menu search response time under 300ms; checkout under 2 seconds	P99 < 300ms / 2s
Security	PCI DSS compliance for all payment card data; OAuth 2.0 for auth	Zero card data stored on platform
Availability	System uptime excluding planned maintenance windows	99.9% uptime SLA
Maintainability	Independent team ownership; deploy one service without touching others	< 1 hour deploy per service

Team Consensus: All three team members agreed on the Online Food Delivery System as the case study domain and on the module assignment above.

3. Use Case Diagram [REQ-02]

3.1. System Actors

The following external actors interact with the Online Food Delivery System:

Actor	Type	Description
Customer	Human (Primary)	Browses restaurants, places orders, processes payments, and tracks delivery status.
Admin	Human	Manages restaurant listings and the delivery fleet, and monitors system analytics.
Delivery Driver	Human (External)	Manages restaurant listings and the delivery fleet, and monitors system analytics.
Payment Gateway (Stripe)	External System	Responsible for card tokenization, processing charges, and handling refunds.
Email / Notification Service	External System	Dispatches order confirmations and real-time status updates to users.

3.2. Use Case Diagram

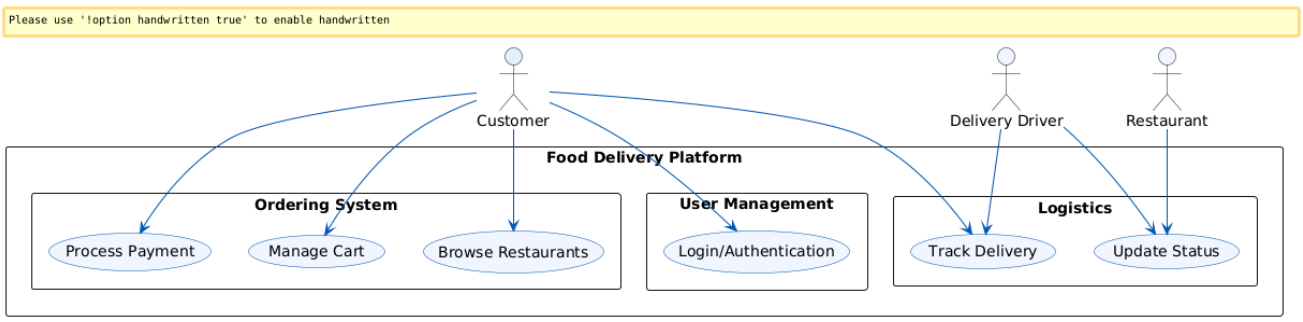


Figure 2.1 — Use Case Diagram: Online Food Delivery System [REQ-02]

The Online Food Delivery System facilitates a seamless interaction between four primary actors—the Customer, Admin, Delivery Driver, and external systems like the Payment Gateway and Email Service—to manage the end-to-end food ordering process. Customers can register, browse restaurant menus, and manage their carts to place orders, which triggers secure processing through the Payment Gateway and automated notifications via the Email Service. Simultaneously, the Admin oversees the platform by managing restaurant listings and delivery fleets while monitoring system analytics, and the Delivery Driver interacts with the system to track and fulfill active deliveries. This integrated workflow ensures that every stage, from initial user authentication to final delivery tracking, is captured within a centralized and efficient digital ecosystem.

3.3. Actor-Use Case Associations Summary

The Online Food Delivery System coordinates interactions between Customers, Admins, Delivery Drivers, and external services like Payment Gateways and Email Services. Customers manage orders and payments, while Admins oversee restaurant listings and delivery fleets. Drivers fulfill active deliveries, creating an integrated digital ecosystem that manages everything from user authentication to final order tracking.

4. Use Case Specifications [REQ-03]

4.1. Use Case Specification — User Login / Registration | Module 1 | Mohammed Alsakkaf

Use Case Name	User Login / Registration
Actors	Customer (primary), AuthService, UserDB

Description	A new or returning customer accesses the platform by either registering a new account with personal details and credentials or logging into an existing account using email and password. The system validates credentials, issues a JWT session token, and grants access to the platform's features.
Pre-Conditions	• Customer has access to the web or mobile application • For login: the customer has a registered account • Email format is valid • System (AuthService and UserDB) is operational
Post-Conditions	• Customer is authenticated and holds a valid JWT session token • User session is recorded in the database • Customer is redirected to the home/dashboard page • For new registration: user record created in UserDB
Main Flow	**1.** Customer navigates to the Login / Register page **2.** System displays login form (email, password) and a 'Register' link **3.** Customer enters email address and password **4.** System validates email format and password length (min 8 characters) **5.** System calls AuthService.authenticate(email, password) **6.** AuthService calls UserDB.findUserByEmail(email) to retrieve user record **7.** UserDB returns the user record with stored password hash **8.** AuthService calls verifyPasswordHash(inputPassword, storedHash) to validate credentials **9.** AuthService calls UserDB.createSession(userId) to record the session **10.** UserDB returns a unique session token **11.** AuthService generates a signed JWT token embedding userId and role **12.** System stores JWT in an HttpOnly cookie on the client

	13. System redirects customer to the restaurant browsing home page **14.** System displays a personalised greeting: 'Welcome back, [Name]'
Alternative Flows	**AF1 — Invalid Credentials:** **At step 8,** verifyPasswordHash returns false. System displays 'Incorrect email or password'. Attempt counter incremented. After 5 failed attempts, account temporarily locked for 15 minutes **AF2 — Account Does Not Exist (Register):** **At step 6,** findUserByEmail returns null. System presents the registration form. Customer enters name, email, password, phone. System calls AuthService.register(dto), creates user record, and proceeds from step 11. **AF2 — Email Already Registered:** During registration, at UserDB.save(), a duplicate email constraint exception is thrown. System displays 'This email is already registered. Please log in instead.'

4.2. Use Case Specification — Browse Restaurants & Menu | Module 1 | Mohammed Alsakkaf

Use Case Name	Browse Restaurants & Menu
Actors	Customer (primary), RestaurantService, RestaurantDB
Description	An authenticated customer searches for nearby restaurants by location and filters by cuisine type and rating. Upon selecting a restaurant, the customer views its full menu, organised by category, with item prices and availability status. The system retrieves and renders data efficiently from a caching

	layer backed by the RestaurantDB.
Pre-Conditions	• Customer is authenticated with a valid session token • Customer's location is available (GPS or manually entered postcode) • RestaurantService and RestaurantDB are operational • At least one restaurant is available in the customer's delivery radius
Post-Conditions	• A list of restaurants matching filters is displayed to the customer • Customer has viewed the menu for at least one restaurant • No data mutation occurs; this is a read-only operation
Main Flow	**1.** Customer arrives at the home page after login **2.** System automatically detects or prompts for customer's current location **3.** Customer optionally applies filters: cuisine type, minimum rating, max delivery time **4.** System calls RestaurantService.searchRestaurants(location, cuisine, rating) **5.** RestaurantService calls RestaurantDB.queryRestaurants(filters) or checks cache **6.** RestaurantDB returns a list of matching restaurant records **7.** RestaurantService maps records to RestaurantDTO[] and returns to WebApp **8.** WebApp renders the restaurant listing (name, rating, distance, estimated delivery time) **9.** Customer selects a specific restaurant to view its menu **10.** System calls RestaurantService.getMenuItems(restaurantId) **11.** RestaurantService calls RestaurantDB.fetchMenu(restaurantId) **12.** RestaurantDB returns all menu categories and items with availability flags

	13. RestaurantService returns MenuItemDTO[] to WebApp **14.** WebApp renders the menu page, grouping items by category **15.** Customer reviews menu items, descriptions, prices, and dietary labels
Alternative Flows	**AF1 —** No Restaurants Found: At step 6, the query returns an empty list. System displays 'No restaurants found for your filters. Try expanding your search radius.' Customer can reset filters. **AF2 —** Restaurant Temporarily Closed: Restaurant is listed but marked isOpen = false. System shows a 'Currently Closed' badge. Customer can browse menu but cannot add to cart until restaurant ope**ns.**

4.3. Use Case Specification — Add Items to Cart | Module 2 | Anas Ahmed Hisham

Use Case Name	Add Items to Cart
Actors	Customer (primary), CartService, InventoryService
Description	The customer selects a food item from the restaurant menu, specifies the quantity, and adds it to their shopping cart. The system verifies item availability through InventoryService, creates or updates a CartItem entity, recalculates the cart total, and updates the UI display. The cart is associated with the customer's session.

Pre-Conditions	• Customer is authenticated and logged in • Customer is viewing a restaurant menu page • The selected menu item is marked as available (isAvailable = true) • The restaurant is currently open (isOpen = true) • CartService and InventoryService are operational
Post-Conditions	• CartItem is persisted in the cart associated with the customer • Cart total (subtotal) is recalculated and updated • Cart item count in the UI header is updated • If item already existed in cart, quantity is incremented rather than creating duplicate
Main Flow	**1.** Customer clicks 'Add to Cart' on a menu item **2.** System displays a quantity selector with a default value of 1 **3.** Customer sets the desired quantity and confirms **4.** System calls CartService.addItem(customerId, menuItemId, quantity) **5.** CartService calls InventoryService.checkAvailability(menuItemId, quantity) **6.** InventoryService confirms stock is sufficient and returns true **7.** CartService calls createCartItem(menuItemId, quantity, unitPrice) **8.** CartService checks if the same menuItemId already exists in the cart **9.** If not a duplicate: CartService persists new CartItem to CartRepository **10.** SCartService calls updateCartTotal() to recalculate subtotal **11.** CartService returns updated CartDTO (items list, subtotal) to WebApp **12.** WebApp updates the cart badge count and displays a success notification **13.** Customer may continue browsing or navigate to cart
Alternative Flows	**AF1 —** Item Unavailable: **At step 6,** InventoryService returns available = false. System shows 'Sorry,

	this item is currently unavailable.' Cart is not modified. **AF2 —** Duplicate Item: **At step 8,** a CartItem for the same menuItemId already exists. CartService increments the existing item's quantity by the requested amount and updates the subtotal. **AF2 —** Invalid Quantity (< 1 or > 20): CartItem.isValid() returns false. System displays 'Please enter a quantity between 1 and 20.' Cart remains unchanged.

4.4. Use Case Specification — Place Order & Confirm | Module 2 | Anas Ahmed Hisham

Use Case Name	Place Order & Confirm
Actors	Customer (primary), CartService, InventoryService, OrderService, NotificationService
Description	The customer reviews their cart, selects or enters a delivery address, and submits the order. The system validates the cart, reserves inventory, creates an Order record with status PENDING_PAYMENT, and asynchronously sends an order acknowledgment notification. The customer is then directed to the payment step.

Pre-Conditions	• Customer is authenticated • Cart contains at least one item • All items are currently available • Customer has a valid delivery address on file or provides a new one • OrderService and InventoryService are operational
Post-Conditions	• Order record created in database with status PENDING_PAYMENT • Inventory reserved for all ordered items (reservation ID stored) • Cart remains in CONVERTED state (no longer modifiable) • Customer is redirected to the payment screen • Asynchronous notification sent confirming order receipt
Main Flow	**1.** Customer navigates to the cart page **2.** System displays all cart items, quantities, prices, and calculated subtotal **3.** Customer clicks 'Proceed to Checkout' **4.** System prompts customer to confirm or enter delivery address **5.** Customer selects delivery address and proceeds **6.** System calls CartService.validateCart(cartId) **7.** CartService calls InventoryService.reserveInventory(cartItems) and receives reservationId **8.** CartService calls OrderService.createOrder(cartId, address, reservationId) **9.** OrderService creates an Order record with status PENDING_PAYMENT **10.** OrderService returns orderId and totalAmount to CartService **11.** CartService marks cart status as CONVERTED **12.** System returns OrderSummaryDTO to WebApp **13.** WebApp displays order confirmation page with orderId and amount

	14. CartService asynchronously calls NotificationService.sendOrderConfirmation(orderId, customerId) **15.** System redirects customer to Payment Module to complete payment
Alternative Flows	**AF1 —** Item Out of Stock at Checkout: **At step 7,** InventoryService throws InsufficientStockException. System notifies customer which item is unavailable. Customer must remove that item or reduce quantity before proceeding. **AF2 —** Empty Cart: **At step 6,** CartService.validateCart() returns isEmpty = true. System redirects customer back to browse page with message 'Your cart is empty.'

4.5. Use Case Specification — Process Payment | Module 3 | Imtiaj

Use Case Name	Process Payment
Actors	Customer (primary), PaymentService, Payment Gateway (Stripe — external), DeliveryService
Description	The customer submits payment card details (via a secure tokenised form) for the confirmed order. PaymentService validates the request, forwards a charge to the external Payment Gateway, records the resulting transaction, updates the order status, and asynchronously triggers delivery dispatch. The customer receives an on-screen payment confirmation.

Pre-Conditions	• Customer holds a valid session and an order with status PENDING_PAYMENT • PaymentService and Payment Gateway are operational • Order amount is confirmed and non-zero • Customer's browser has loaded the PCI-compliant card entry frame
Post-Conditions	• Transaction record created in database with status APPROVED • Order status updated to CONFIRMED • DeliveryService asynchronously begins driver assignment • Customer sees payment success screen • Payment receipt sent to customer's email (asynchronous)
Main Flow	**1.** Customer enters card details in the tokenised payment iframe **2.** Customer clicks 'Pay Now' **3.** System calls PaymentService.processPayment(orderId, cardToken, amount) **4.** PaymentService calls validatePaymentRequest(amount, cardToken) internally **5.** PaymentService calls PaymentGateway.chargeCard(cardToken, amount, 'MYR') **6.** Payment Gateway processes the charge and returns transactionId with status APPROVED **7.** PaymentService calls recordTransaction(orderId, transactionId) to persist the record **8.** PaymentService returns PaymentConfirmation to WebApp **9.** WebApp displays 'Payment Successful — Order #[orderId] confirmed' **10.** PaymentService asynchronously calls DeliveryService.dispatchDelivery(orderId, restaurantId, address)

	11. OrderService updates order status to CONFIRMED **12.** NotificationService sends payment receipt email to customer asynchronously
Alternative Flows	**Payment Declined** 1. The system detects insufficient funds or invalid cards. 2. The system notifies the user of the failure. 3. Users are prompted to try a different payment method.

4.6. Use Case Specification — Track Delivery | Module 3 | Imtiaj

Use Case Name	Track Delivery in Real Time
Actors	Customer (primary), DeliveryService, TrackingService, Delivery Driver
Description	After payment is confirmed, the customer can view the real-time location of their assigned delivery driver on an interactive map. TrackingService maintains a WebSocket connection, pushing GPS coordinate updates every 10 seconds. The customer sees the driver's current position, estimated arrival time, and order status transitions (Preparing → Out for Delivery → Delivered).
Pre-Conditions	• Payment has been confirmed and order status is CONFIRMED • A delivery driver has been assigned to the order • TrackingService and WebSocket infrastructure are operational

	• Customer has the tracking page open in their browser
Post-Conditions	• ETA is refreshed dynamically as the driver moves • Order status transitions are reflected immediately • On delivery: order status updated to DELIVERED, customer prompted to rate the driver
Main Flow	**1.** Customer navigates to the order tracking page **2.** System calls DeliveryService.getDeliveryStatus(orderId) **3.** DeliveryService calls TrackingService.queryDeliveryStatus(orderId) **4.** TrackingService calls TrackingService.getDriverLocation(orderId) to fetch GPS coordinates **5.** TrackingService calls TrackingService.getETA(orderId) to calculate estimated arrival **6.** TrackingService returns DeliveryStatusDTO (status, GPS coords, ETA) to DeliveryService **7.** DeliveryService returns TrackingInfo to WebApp **8.** WebApp renders the live map with driver marker and ETA countdown **9.** TrackingService establishes a WebSocket connection with the customer's browser **10.** Every 10 seconds, Delivery Driver app sends GPS update to TrackingService **11.** TrackingService calls broadcastLocation(orderId, gpsCoords) via WebSocket push to customer **12.** WebApp smoothly animates the driver marker to the new position **13.** On delivery completion, Delivery Driver marks order as DELIVERED **14.** System updates order status to DELIVERED in database **15.** Customer receives an in-app notification: 'Your order has been

	delivered!' **16.** System prompts customer to rate the driver (1–5 stars)
Alternative Flows	**A1: GPS Signal Lost** 1. The system cannot retrieve the driver's real-time coordinates. 2. The system displays the "Last Known Location" to the User. 3. The system attempts to reconnect to the Driver's device.

5. Sequence Diagrams [REQ-04]

5.1. Sequence Diagram — User Login & Browse Restaurants | Module 1 | Mohammed Alsakkaf

The diagram below illustrates the message flow for two use cases in Module 1. The Login Flow (blue region) shows the Customer authenticating through WebApp → AuthService → UserDB, with password verification and JWT token generation. The Browse Flow (green region) shows the Customer searching for restaurants through WebApp → RestaurantService → RestaurantDB, then selecting a restaurant to view its menu. All method calls in this diagram correspond directly to methods defined in the Module 1 UML Class Diagram (Section 7.1).

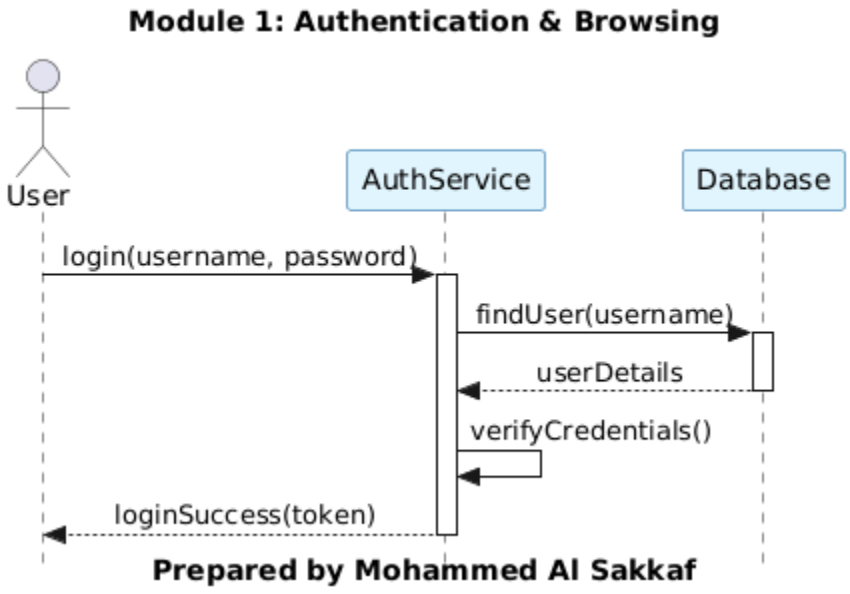


Figure 5.1 — Sequence Diagram: User Login & Browse Restaurants — Module 1 — Mohammed Alsakkaf

This sequence diagram depicts Module 1's functional flow, starting with the Customer logging into the WebApp via the Authentication Service to establish a secure session. Once authenticated, the Customer submits search criteria like location or cuisine, prompting the WebApp to query the RestaurantService. The service pulls filtered restaurant and menu data from the database and returns it to the WebApp, which then renders the final menu page for the user.

5.2. Sequence Diagram — Add to Cart & Place Order | Module 2 | Anas Ahmed Hisham

The diagram illustrates the Add to Cart Flow (green region) where CartService coordinates with InventoryService to verify availability before persisting a CartItem, and the Place Order Flow (yellow region) where CartService orchestrates inventory reservation and Order creation through OrderService. The asynchronous notification call (purple dashed arrow) models the non-blocking email dispatch. The alt frame at the bottom captures the InsufficientStockException alternative flow from Use Case 3.4.

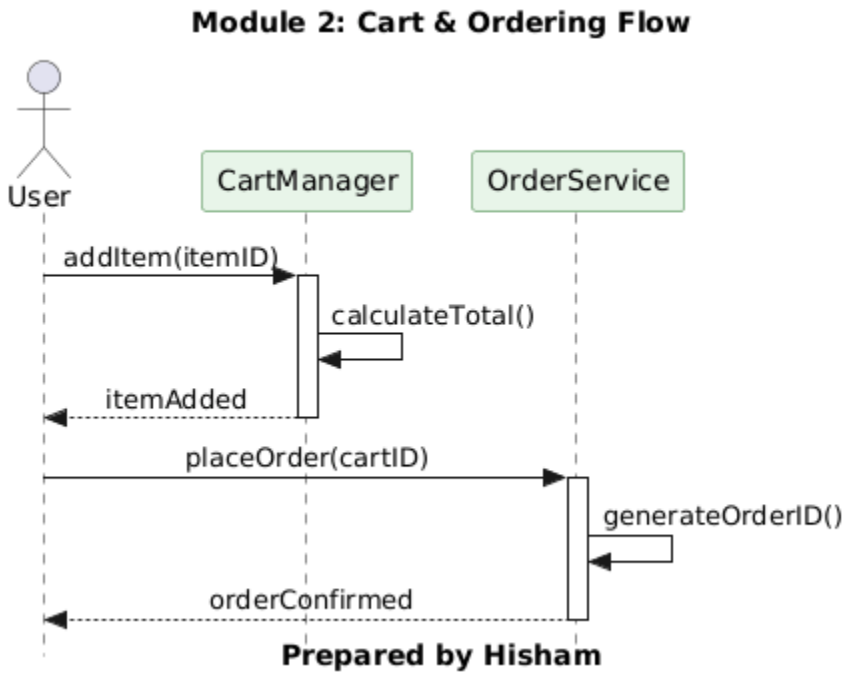


Figure 5.2 —Sequence Diagram: Add to Cart & Place Order — Module 2 — Anas Ahmed Hisham

This sequence diagram for Module 2 illustrates the Customer adding items to a basket, where the CartService manages totals and persistence. When an order is placed, the OrderService validates availability and customer details before the PaymentService authorizes the transaction. Following successful payment,

the system generates an order ID, records the transaction, and notifies the Customer, finalizing the transition from cart to order fulfillment.

5.3. Sequence Diagram — Process Payment & Track Delivery | Module 3 | Imtiaj

The diagram shows the Payment Flow (red region) where PaymentService delegates the charge to the external Payment Gateway, records the transaction, and asynchronously dispatches delivery. The Track Delivery Flow (green region) shows how DeliveryService and TrackingService serve the initial status request, followed by a persistent WebSocket push (purple dashed arrow) that streams location updates directly to the customer's browser. This pattern ensures near-real-time driver tracking without polling overhead.

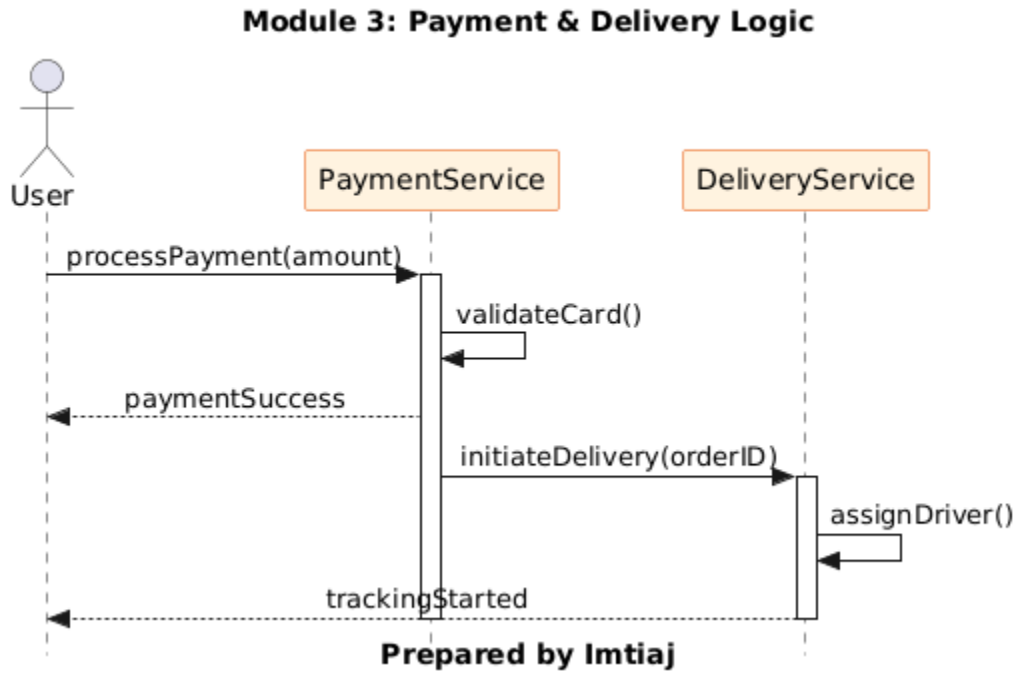


Figure 5.3 — Sequence Diagram: Process Payment & Track Delivery — Module 3 — Imtiaj

This sequence diagram for Module 3 outlines the Customer submitting payment via the WebApp, triggering authorization through the Payment Gateway. Once approved, the PaymentService notifies the Delivery Service to dispatch the order while the TrackingService retrieves real-time coordinates from the driver. This process provides the Customer with live status updates and estimated arrival times, ensuring full visibility from transaction settlement until final delivery completion.

6. Architectural Style [REQ-05]

6.1. Selected Architectural Style: Microservices Architecture

The team selected Microservices Architecture as the overarching structural style for the Online Food Delivery System. In this style, the system is decomposed into a collection of small, independently deployable services — each owning its own data store, business logic, and API surface. Services communicate over well-defined interfaces (REST/HTTP for synchronous calls; an event bus for asynchronous, non-blocking operations such as notifications and delivery dispatch).

6.2. Rationale — Why Microservices?

The selection was driven by a direct mapping of the system's non-functional requirements to the architectural capabilities of Microservices:

- **Scalability:** During lunch and dinner rush periods, restaurant search traffic spikes dramatically. In a monolithic architecture, scaling the search functionality requires replicating the entire application, wasting resources. With Microservices, RestaurantService can be independently auto-scaled (e.g., from 3 to 15 instances in 90 seconds) without touching PaymentService or TrackingService.
- **Fault Isolation:** A failure in DeliveryService (e.g., a GPS data provider outage) should not cascade to PaymentService or prevent customers from browsing restaurants. Microservices achieve this through isolated failure boundaries; each service has its own circuit breaker and fallback behaviour.
- **Team Autonomy & Maintainability:** Each module in this assignment maps directly to an independently ownable microservice (User Auth/Restaurant, Cart/Order, Payment/Delivery). Teams can release new versions of their service without coordinating a full system deployment, reducing release cycle time from weeks to hours.
- **Security Segmentation:** PCI DSS compliance requires that payment card data be handled in an isolated, hardened environment. Microservices allows PaymentService to operate in a dedicated PCI-scoped network segment with stricter access controls, without imposing those restrictions on the entire platform.

6.3. Mapping to Functional Requirements

Actor	Microservice	Architectural Benefit
User Login / Register	Auth Service	Stateless JWT; independently secured
Browse Restaurants & Menu	Restaurant Service	Cache layer; auto-scale for search load
Add to Cart / Place Order	Cart & Order Service	Eventual consistency; saga pattern for rollback
Process Payment	Payment Service	PCI-scoped network; isolated blast radius
Track Delivery	Delivery & Tracking Service	WebSocket gateway; event-driven GPS push
Manage Restaurants (Admin)	Admin Service	Low traffic; no scaling pressure

6.4. Trade-offs and Mitigations

Trade-off	Impact	Impact
Distributed complexity	Cross-service debugging and tracing is harder than a monolith	Distributed tracing (Zipkin/Jaeger); centralised logging (ELK stack)
Network latency between services	Additional hop per service call adds ~2–10ms	Service mesh (Istio); co-locate high-coupling services in same AZ
Data consistency across services	No single database transaction spans services	Saga pattern for multi-step workflows (e.g., Order + Payment + Inventory)
Operational overhead	More services = more deployment pipelines and monitoring	Kubernetes orchestration; CI/CD pipelines per service

7. UML Structural Diagrams [REQ-06]

7.1. Class Diagram — User Auth & Restaurant Browsing Module | Module 1 | Mohammed Alsakkaf

This class diagram for Module 1 models primary entities like User and AuthService, using the Repository Pattern to decouple identity data from persistence via the UserRepository interface. RestaurantService manages Restaurant objects and their composed MenuItem entities, ensuring that menu lifecycle is tied to the restaurant instance. By utilizing interfaces for all external dependencies, the architecture maintains loose coupling consistent with microservices principles.

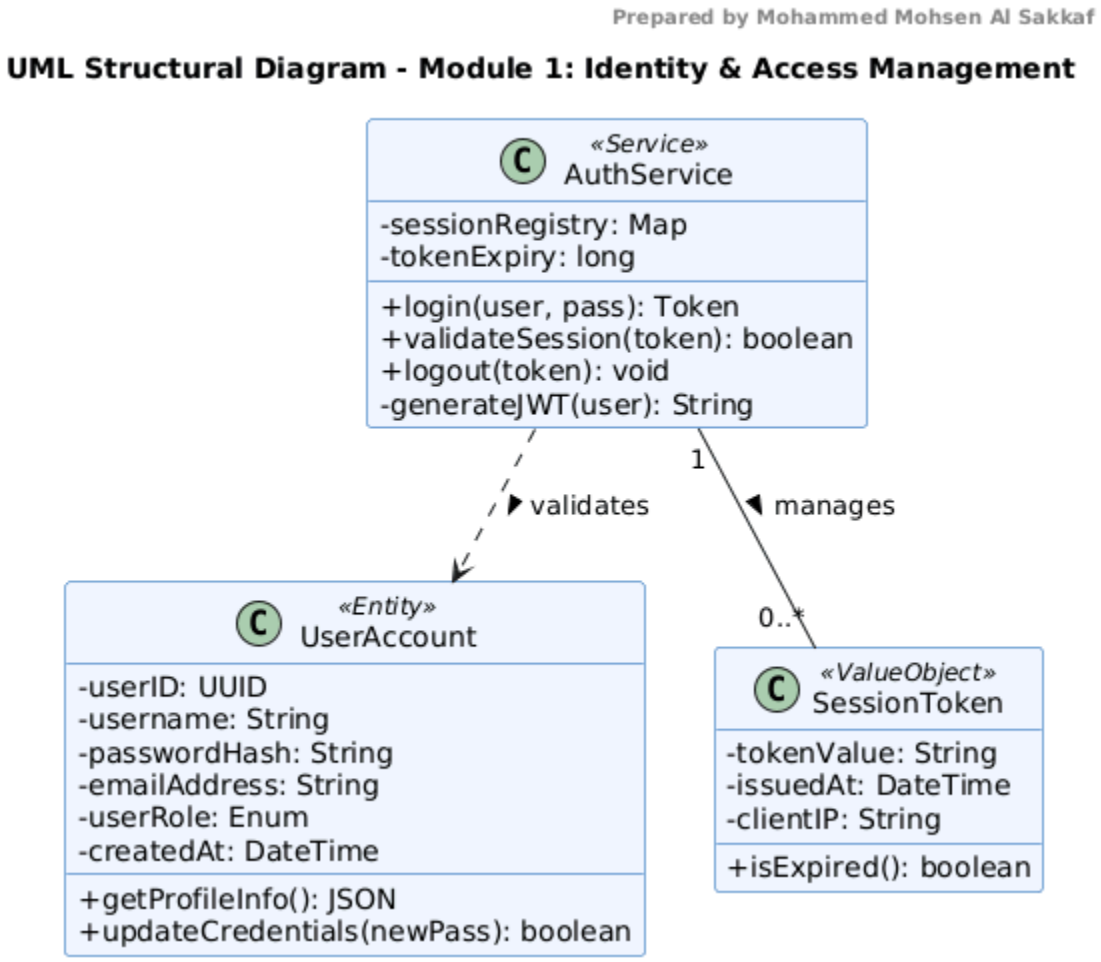


Figure 7.1 — Class Diagram: User Auth & Restaurant Browsing Module — Module 1 — Mohammed Alsakkaf

This class diagram for Module 1 defines the structural blueprint for user authentication and restaurant discovery via User and Restaurant entities. It utilizes a repository pattern with UserRepository and RestaurantRepository to decouple business logic from data access, standardizing methods like findByEmail and searchRestaurants. Within the module, the AuthService manages session security while

the RestaurantService coordinates MenuItem retrieval, establishing a robust foundation for data integrity and core user-database interactions.

7.1.1. Coherence Validation — Module 1

Sequence Message (Fig 5.1)	UML Method (Fig 7.1)	Match
`authenticate(email, password)`	`AuthService.authenticate()`	✓
`findUserByEmail(email)`	`UserRepository.findByEmail()`	✓
`verifyPasswordHash(p,h)`	`AuthService.verifyPasswordHash()`	✓
`createSession(userId)`	`AuthService.createSession()`	✓
`searchRestaurants(location,cuisine)`	`RestaurantService.searchRestaurants()`	✓
`getMenuItems(restaurantId)`	`RestaurantService.getMenuItems()`	✓
`queryRestaurants(filters)`	`RestaurantService.queryRestaurants()`	✓
`fetchMenu(restaurantId)`	`RestaurantService.fetchMenu()`	✓

7.2. Class Diagram — Shopping Cart & Order Placement Module | Module 2 | Anas Ahmed Hisham

The class diagram for Module 2 features the CartService as the central coordinator, utilizing the CartRepository interface and InventoryService for loose coupling. ShoppingCart maintains a composition relationship with CartItem, ensuring items only exist within a cart context. CartRepositoryImpl provides concrete persistence, while the OrderService and Order classes manage the downstream lifecycle. InventoryService acts as an interface boundary, facilitating cross-module communication consistent with microservices architecture.

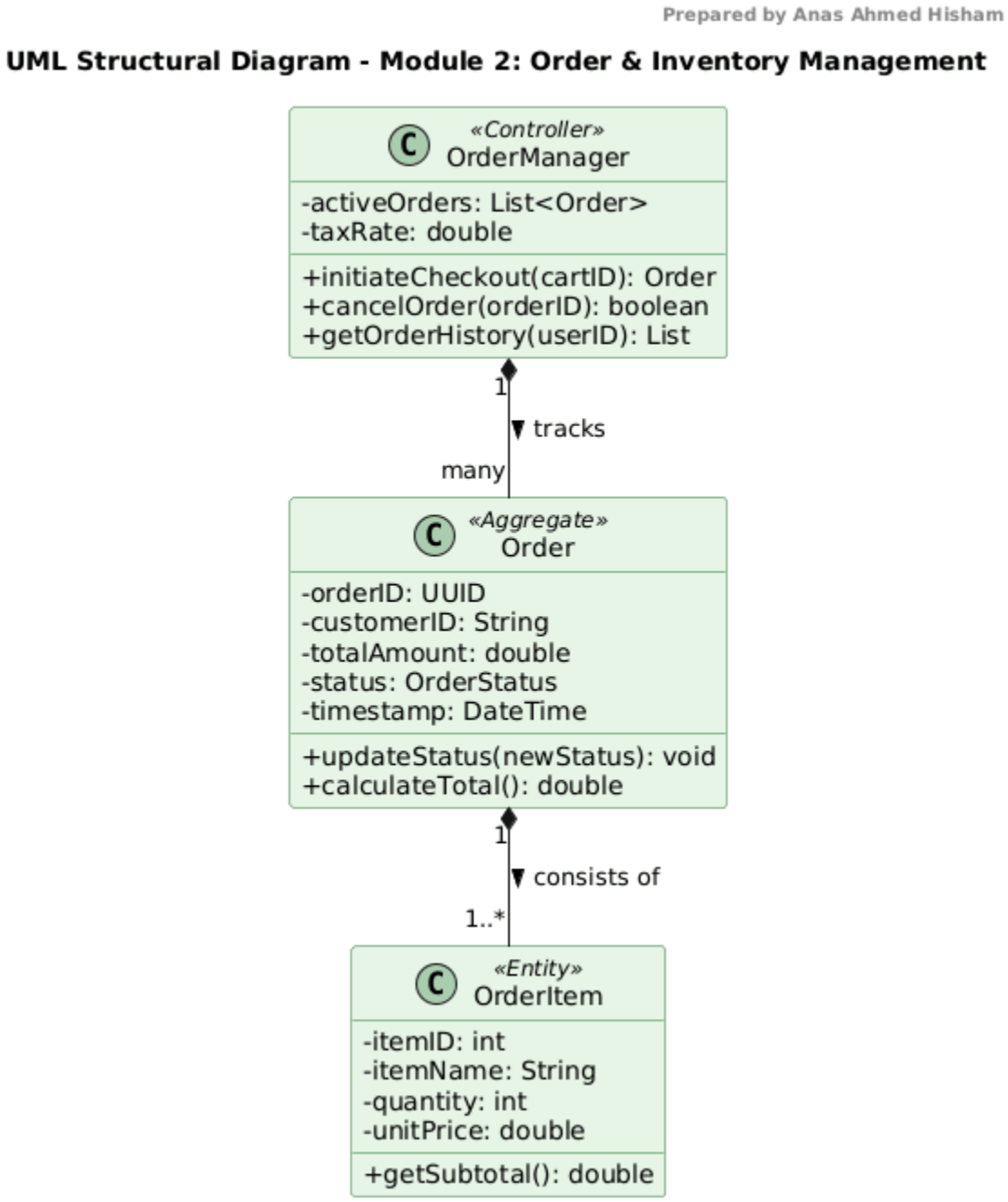


Figure 7.2 — Class Diagram: Shopping Cart & Order Placement Module — Module 2 — Anas Ahmed Hisham

This class diagram for Module 2 defines the relationship between CartService, ShoppingCart, and OrderService to manage the transition from selection to order creation. Centered on the ShoppingCart, which aggregates CartItem entities, the system validates inventory via InventoryService. Upon finalization, CartService directs OrderService to transform cart contents into a persistent Order entity using CartRepository and OrderRepository. This architecture ensures separation of concerns, independent scaling, and robust data validation across all transactions.

7.2.1. Coherence Validation — Module 2

Sequence Message (Fig 5.2)	UML Method (Fig 7.2)	Match
`addItem(custId, menuItemId, qty)`	`CartService.addItem()`	✓
`checkAvailability(menuItemId, qty)`	`InventoryService.checkAvailability()`	✓
`createCartItem(menuItemId,qty,p)`	`createCartItem(menuItemId,qty,p)`	✓
`updateCartTotal()`	`CartService.updateCartTotal()`	✓
`validateCart(cartId)`	`CartService.validateCart()`	✓
`reserveInventory(cartItems)`	`CartService.reserveInventory()`	✓
`createOrder(cartId, addr, rsvId)`	`OrderService.createOrder()`	✓
`sendOrderConfirmation(orderId,custId)`	`OrderService.sendOrderConfirmation()`	✓

7.3. Class Diagram — Payment Processing & Delivery Tracking Module | Module 3 | Imtiaj

This class diagram for Module 3 illustrates the structural design for financial transactions and logistics via the PaymentService and DeliveryService. PaymentService coordinates with PaymentGateway and StripeGatewayImpl to authorize charges and record Transaction entities for financial clearance. Once validated, DeliveryService manages order assignments to the Driver class and interfaces with TrackingService for geographic updates. By utilizing DeliveryStatusDTO and TransactionDTO, the system separates internal logic from shared data structures, ensuring a scalable framework for processing payments and monitoring delivery progress.

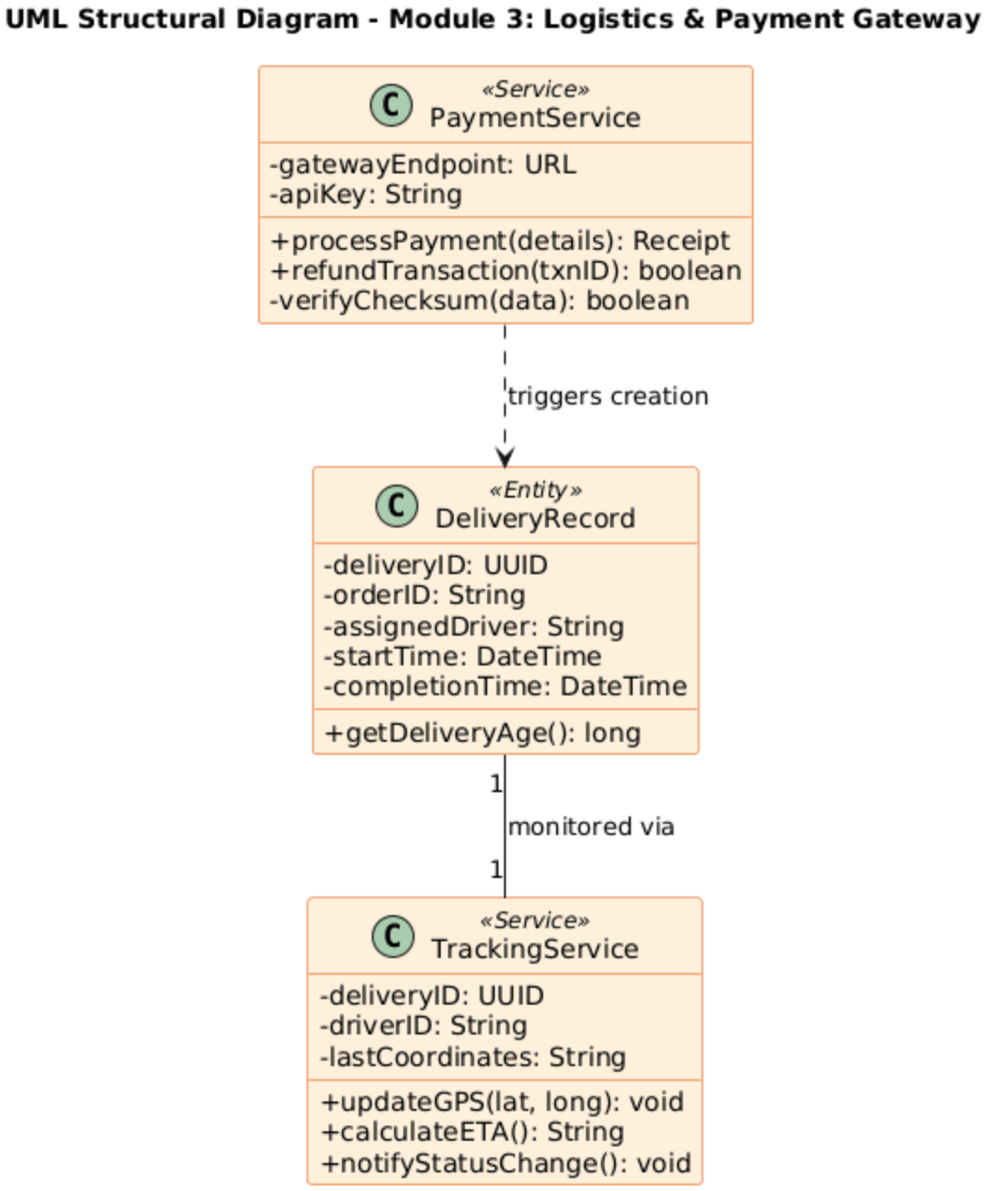


Figure 7.3 — Class Diagram: Payment Processing & Delivery Tracking Module — Module 3 — Imtiaj

This class diagram for Module 3 illustrates the structural design for financial transactions and logistics via the PaymentService and DeliveryService. PaymentService coordinates with PaymentGateway and StripeGatewayImpl to authorize charges and record Transaction entities for financial clearance. Once validated, DeliveryService manages order assignments to the Driver class and interfaces with TrackingService for geographic updates. By utilizing DeliveryStatusDTO and TransactionDTO, the system separates internal logic from shared data structures, ensuring a scalable framework for processing payments and monitoring delivery progress.

Architectural Coherence Mapping Table

Sequence Diagram Label (5.3)	Actual Class Method (7.3)	Purpose
Process	+ processPayment()	Executes the transaction logic.
Verify	+ validateCard()	Checks card validity with the bank.
Track	+ updateLocation()	Updates GPS coordinates in the DB.
Status	+ getETAToUser()	Calculates delivery time for the UI.

7.3.1. Coherence Validation — Module 3

Sequence Message (Fig 5.3)	UML Method (Fig 7.3)	Match
`processPayment(orderId,token,amt)`	`PaymentService.processPayment()`	✓
`validatePaymentRequest(amt,token)`	`PaymentService.validatePaymentRequest()`	✓
`chargeCard(token, amount, cur)`	`PaymentGateway.chargeCard()`	✓
`recordTransaction(orderId,txId)`	`PaymentService.recordTransaction()`	✓
`dispatchDelivery(orderId,rstId,addr)`	`DeliveryService.dispatchDelivery()`	✓
`getDeliveryStatus(orderId)`	`DeliveryService.queryDeliveryStatus()`	✓
`getDriverLocation(orderId)`	`TrackingService.getDriverLocation()`	✓
`broadcastLocation(orderId,gps)`	`TrackingService.broadcastLocation()`	✓

8. Conclusion & Reflection

8.1. Summary

This report has delivered a complete six-phase software architecture design for the Online Food Delivery System. Beginning with a clearly scoped real-world case study, the team established a shared architectural foundation (Use Case Diagram, Architectural Style) before each member independently detailed their assigned module through use case specifications, interaction sequence diagrams, and UML structural models.

The Microservices Architecture was selected and justified specifically against the system's non-functional requirements — most critically, the need for independent scalability of the restaurant search service during peak traffic, fault isolation between payment processing and other services, and team-level module ownership. Trade-offs including distributed complexity and eventual consistency were acknowledged and mitigated through concrete patterns (Saga, Repository, WebSocket Gateway).

Architectural coherence was validated for each module by cross-checking that every message call in the sequence diagrams corresponds to a method in the UML class diagrams, and every use case operation maps to at least one UML class method. All three coherence tables
(Sections 7.1.1, 7.2.1, and 7.3.1) confirm full alignment.

8.2. Lessons Learned

- Coherence between phases is non-trivial and must be enforced proactively, not fixed retroactively. Naming methods consistently across specs, sequences, and UML from the beginning avoids expensive rework.
- The Microservices style adds significant value for systems with clear domain boundaries and mixed traffic profiles — but would be over-engineering for a simple internal CRUD application.
- Use case specifications are most valuable when alternative flows are detailed. Alternative flows reveal edge cases that directly influence the design of error handling in sequence diagrams and defensive methods in UML classes.

9. Appendix

9.1. Module Assignment Matrix

Member	Module	Use Case Specifications	Sequence Diagrams	UML Structural Diagrams
Mohammed Alsakkaf	Module 1: User Auth & Restaurant	**Sections 4.1, 4.2**	**Section 5.1**	**Section 7.1**
Anas Ahmed Hisham	Module 2: Cart & Order	**Sections 4.3, 4.4**	**Section 5.2**	**Section 7.2**
Imtiaj	Module 3: Payment & Delivery	**Sections 4.5, 4.6**	**Section 5.3**	**Section 7.3**

9.2. Group Work Contributions

Member Name	Role & Responsibility	Assigned Module(s)	Tasks Completed (REQ 03, 04, 06)	Contribution
Mohammed	Lead Architect & Project Manager	Module 1: Core System & User Management	Use Case Specs, Sequence Diagrams, and Class Diagrams for Core Logic.and dealing with documentation of the entire Report	**44%**
Hisham	System Analyst	Module 2: Order Processing & Tracking	Use Case Specs, Sequence Diagrams, and Class Diagrams for Order Flow.	**28%**
Imtiaj	Interface Designer	Module 3: Payment Gateway & Delivery	Use Case Specs, Sequence Diagrams, and Class Diagrams for Logistics.	**28%**

10. Final Quality Assurance Checklist

Requirement	Status
Are all diagrams clear, legible, and properly labeled with member names?	[x] Yes

Does the report contain individual work (Tasks 3, 4, 6) for all members?	[x] Yes
Is the global architectural style justified against Non-Functional Requirements?	[x] Yes
Does the UML Structural view mathematically align with Sequence Diagrams?	[x] Yes
Is the report consolidated into one single, professional file?	[x] Yes